

REPORT

[VULN-02] CSRF in Change Email → Account Takeover

1. Summary (Tóm tắt)

Chức năng đổi email (`POST /api/user/change-email`) không triển khai cơ chế xác thực CSRF token. Attacker có thể tạo một trang web độc hại chứa form tự động submit, lừa victim truy cập — trình duyệt sẽ tự động đính kèm cookie phiên hợp lệ, khiến server chấp nhận yêu cầu thay đổi email mà không có sự đồng ý của người dùng, dẫn đến Account Takeover.

2. Severity

CVSS Score	Rating	Vector	Version
6.5	Medium	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:L/A:N	CVSS 3.1

Lưu ý: Cookie đã được cấu hình `HttpOnly: true`, do đó attacker không thể đọc trực tiếp token qua JavaScript. Tuy nhiên, CSRF không cần đọc cookie — trình duyệt tự động gửi kèm cookie khi request được gửi từ domain của victim.

3. Affected Component

- **Endpoint:** `POST /api/user/change-email`
- **File:** `UserController.java` (line ~84)
- **Root cause:** Thiếu CSRF token validation — server không kiểm tra origin của request
- **Điều kiện khai thác:** Victim đang đăng nhập và truy cập link/trang độc hại của attacker

4. Description (Mô tả chi tiết)

Lỗi hỏng CSRF (Cross-Site Request Forgery) xảy ra khi server không phân biệt được request hợp lệ từ ứng dụng với request giả mạo từ một trang web bên ngoài.

a) Cơ chế hoạt động của lỗi hỏng

Endpoint `POST /api/user/change-email` xử lý yêu cầu đổi email dựa hoàn toàn vào session cookie để xác thực người dùng, mà không yêu cầu bất kỳ giá trị bí mật nào (CSRF token) được nhúng trong form hoặc header. Do trình duyệt luôn tự động đính kèm cookie vào mọi request gửi đến domain đích — kể cả request xuất phát từ trang web khác — server không có cách nào phân biệt đây là hành động chủ động của người dùng hay yêu cầu bị giả mạo.

b) Thiếu CSRF Token (UserController.java)

Backend nhận `newEmail` từ request body và xử lý trực tiếp mà không kiểm tra:

- `X-CSRF-Token` header
- `SameSite` cookie attribute (hoặc đang set là `None/Lax`)
- Origin/Referer header validation

c) Điều kiện để khai thác thành công

- Victim đang có phiên đăng nhập hợp lệ trên ứng dụng
- Victim truy cập trang web độc hại do attacker kiểm soát (qua link lừa đảo, quảng cáo, email phishing...)
- Trình duyệt tự động gửi session cookie kèm theo request — không cần bất kỳ tương tác phức tạp nào ngoài việc nhấn nút

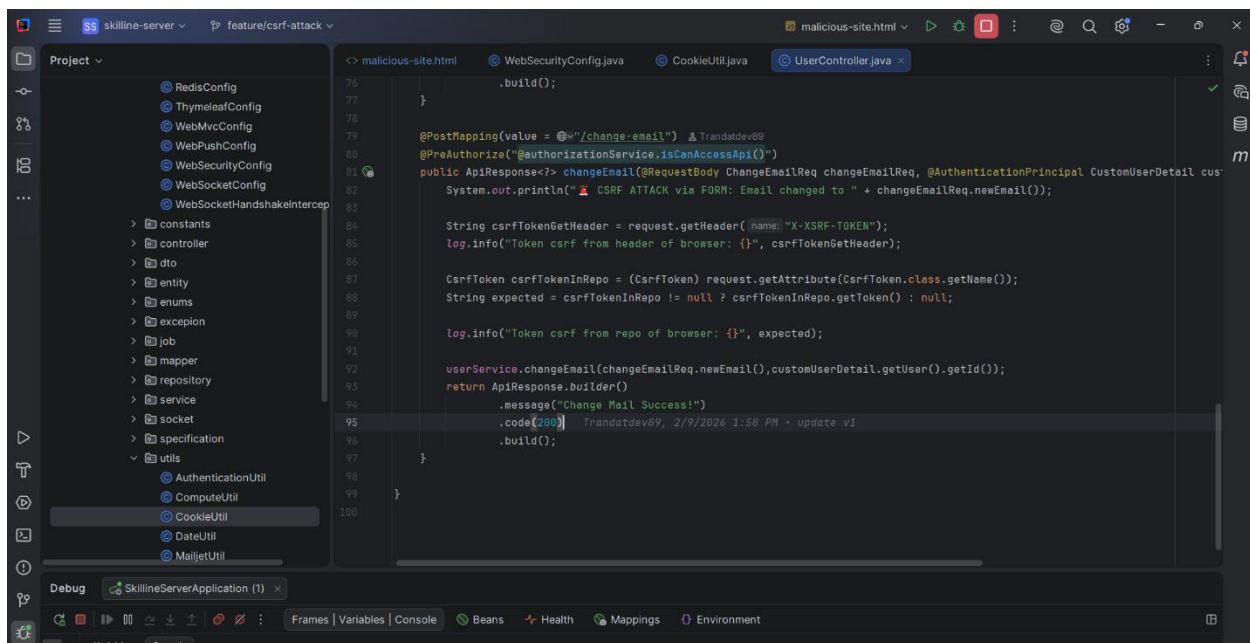
5. Proof of Concept (PoC)

Bước 1 — Môi trường test

- **Backend:** Spring Boot tại `http://localhost:8080`
- **Frontend:** Vue 3 tại `http://localhost:5173`
- **Attacker server:** `http://attacker.local:3000` (trang web giả mạo)
- **Attacker account:** `Tranlong` / `longdeptra12`
- **Victim account:** `tranquocdat` / `longga12`
- **Attacker email nhận về:** `hacker123@evil.com`

Bước 2 — Xác nhận endpoint không có CSRF protection

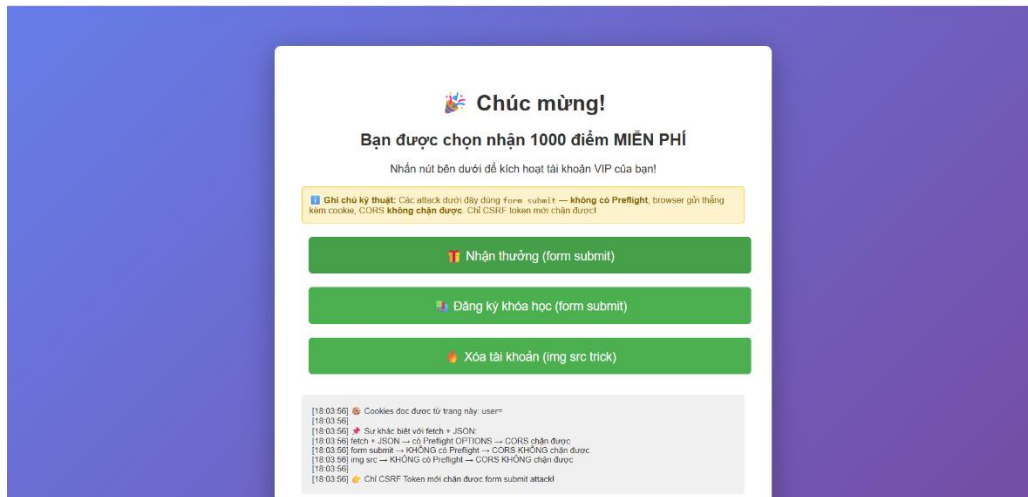
Quan sát thấy request `POST /api/user/change-email` không chứa CSRF token trong header hoặc body — server chỉ dựa vào session cookie để xác thực.



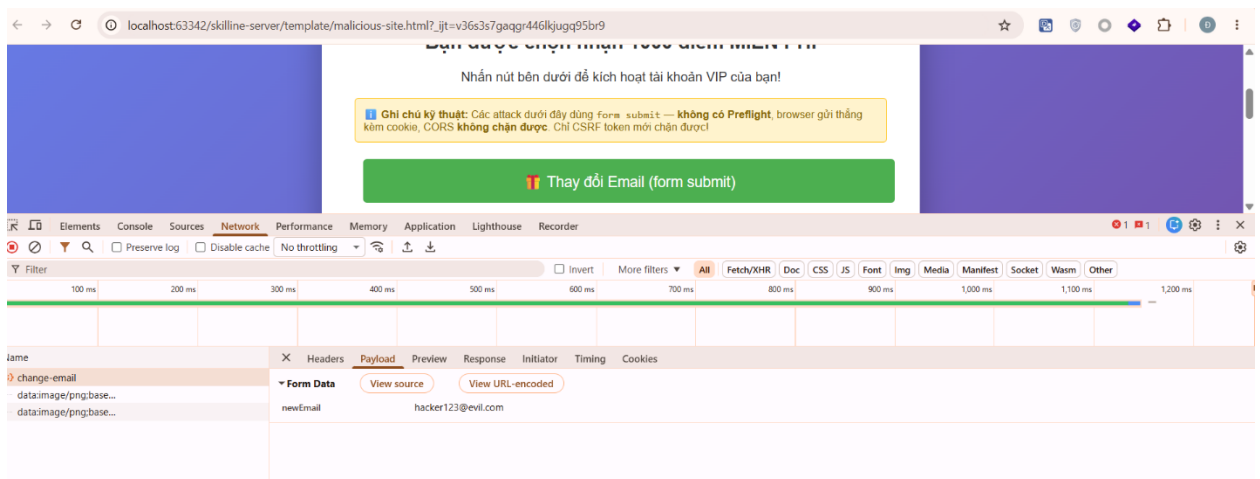
Bước 3 — Tạo trang web độc hại

Attacker tạo file HTML chứa form tự động submit khi victim nhấn nút, file malicious.html:

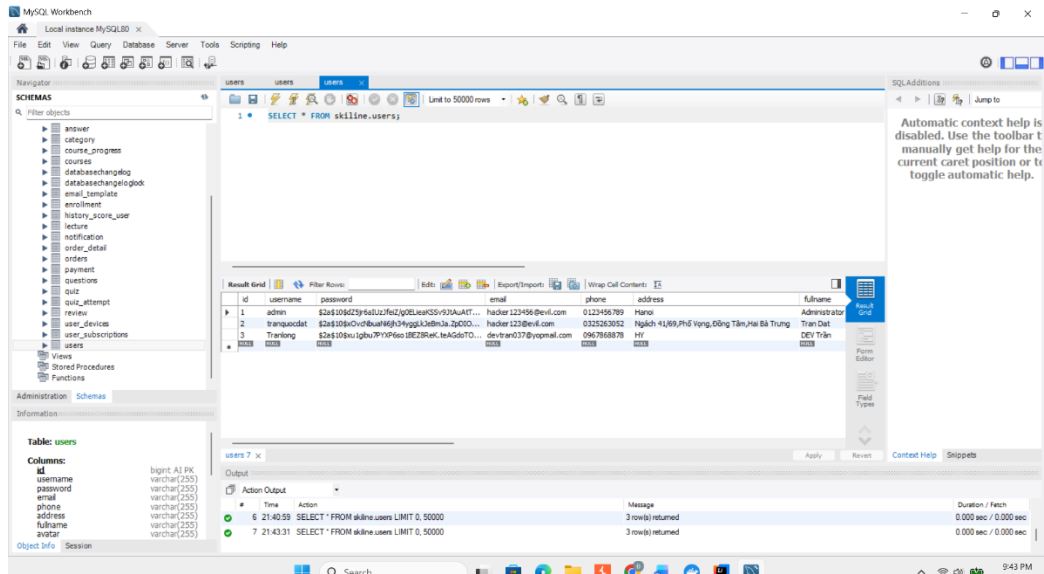
```
<!-- evil.html -->
<form id="form-change-email"
      action="http://localhost:8080/api/user/change-email"
      method="POST"
      enctype="application/x-www-form-urlencoded"
      target="hidden-iframe-1">
  <input type="hidden" name="newEmail" value="hacker123@evil.com">
</form>
```

1. Victim truy cập trang và nhấn nút "Nhận thưởng".
2. Trình duyệt tự động gửi `POST /api/user/change-email` kèm cookie hợp lệ của victim đến server.



3. Server không kiểm tra CSRF token — xử lý request, đổi email của `tranquocdat` thành `hacker123@evil.com`, truy cập database ta sẽ thấy bản ghi của nạn nhân đã bị đổi email



Bước 5 — Chiếm tài khoản (Account Takeover)

Sau khi email của victim đã bị thay đổi thành `attacker@evil.com`, attacker thực hiện chức năng "Quên mật khẩu" — link reset password được gửi về email của attacker. Từ đây attacker đặt lại mật khẩu mới và hoàn toàn chiếm quyền kiểm soát tài khoản `Tranlong`.

6. Impact

Nếu khai thác trên môi trường production, lỗ hổng này dẫn đến:

4. **Account Takeover:** Attacker chiếm toàn quyền tài khoản victim thông qua luồng reset password.
5. **Truy cập & thao túng dữ liệu cá nhân:** Sau khi chiếm tài khoản, attacker có thể đọc, sửa, xóa toàn bộ dữ liệu của victim.
6. **Không để lại dấu vết rõ ràng:** Toàn bộ hành động được thực hiện qua trình duyệt của victim, khó phát hiện qua log thông thường.
7. **Ảnh hưởng uy tín & vận hành:** Người dùng mất quyền truy cập tài khoản mà không rõ nguyên nhân.

7. Remediation

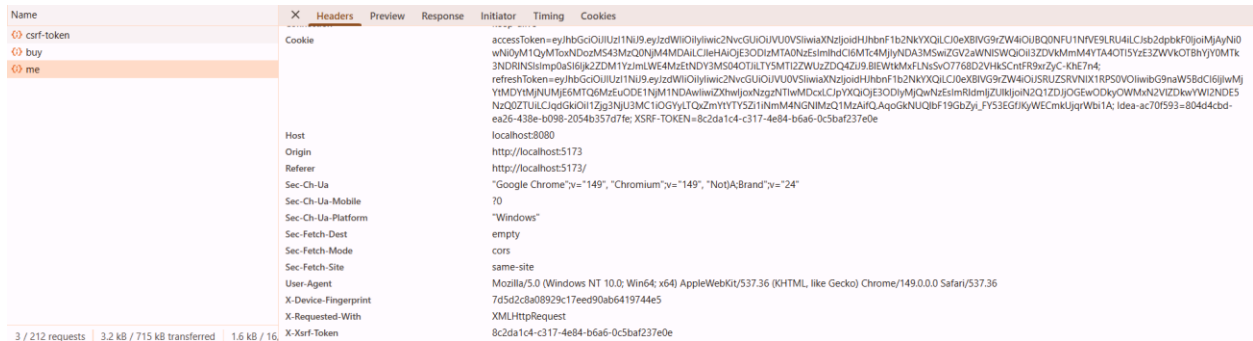
a) Triển khai CSRF Token (Ưu tiên cao nhất)

Sử dụng thư viện Spring Security để tự động sinh và validate CSRF token cho mỗi phiên làm việc. Server chỉ chấp nhận request khi token trong form/header khớp với token trong session.

Trong file `WebSecurityConfig` bật csrf lên(line 106) và viết 1 api `auth/csrf` token để thực hiện get csrf token để trả về cho client và mỗi khi request nào của user được gửi đi thì cookie sẽ gửi kèm theo `x-xsrf-token` ở header và `XSRF-TOKEN` ở cookie.

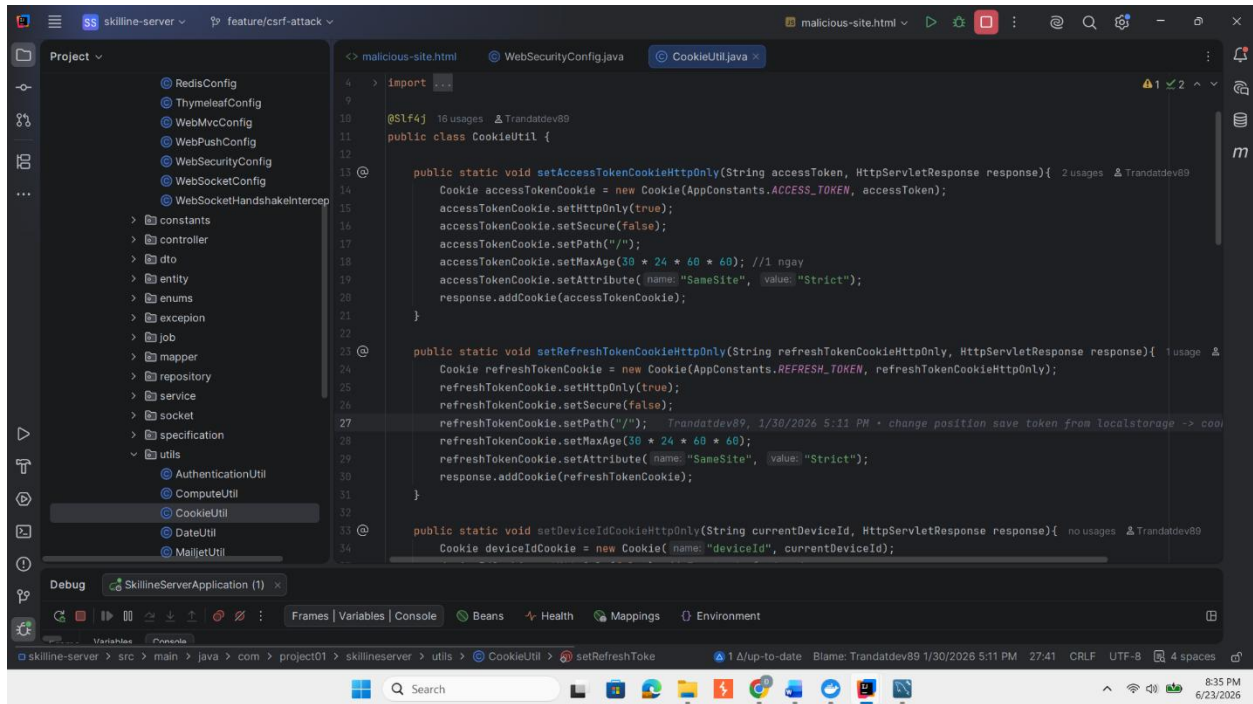
```
105 return httpSecurity
106     .csrf(CsrConfigurer<HttpSecurity> httpSecurityCsrfConfigurer -> httpSecurityCsrfConfigurer
107         .csrfTokenRepository(csrfTokenRepository)
108         .csrfTokenRequestHandler(new CsrfTokenRequestAttributeHandler())
109         .sessionAuthenticationStrategy(new NullAuthenticatedSessionStrategy())
110         .ignoringRequestMatchers(-patterns: "/auth/**", "/api/file/**", "/ws/**",
111             "/api/test", "/api/media/**"))
112     .authorizeHttpRequests(AuthorizationManagerRequestMat... http -> http
113         .requestMatchers(HttpMethod.GET, @"/product/**").permitAll()
114         .requestMatchers(HttpMethod.GET, @"/api/category/pagination").permitAll()
115         .requestMatchers(HttpMethod.GET, @"/review/**").permitAll()
116         .requestMatchers(HttpMethod.GET, @"/uploads/image/**").permitAll()
117         .requestMatchers(HttpMethod.GET, @"/uploads/lecture/**").permitAll()
118         .requestMatchers(@PUBLIC_ENTRYPOINT).permitAll()
119         .anyRequest().authenticated())
120     .formLogin(AbtractHttpConfigurer::disable)
121     .oauth2ResourceServer(OAuth2ResourceServerConfigurer<HttpSecurity> oauth -> oauth
122         .bearerTokenResolver(cookieBearerTokenResolver)
123         .accessDeniedHandler(new JwtAccessDeniedEntryPoint())
124         .authenticationEntryPoint(new JwtAuthenticationEntryPoint())
125         .jwt(JwtConfigurer.config -> config
126             .jwtAuthenticationConverter(customJwtAuthConverter))
```

```
81 .code(200)
82 .build();
83 }
84
85 @PostMapping(value = @="/refresh-token") &Trandatdev89
86 public ApiResponse<?> refreshToken(HttpServletRequest request, HttpServletResponse response) throws ParseException {
87     String refreshToken = CookieUtil.getTokenFromCookie(refreshTokenName, request);
88     String accessToken = CookieUtil.getTokenFromCookie(accessTokenName, request);
89     authService.refreshToken(refreshToken, accessToken, response);
90     return ApiResponse.<=>builder()
91         .code(200)
92         .message("Refresh Token Success!")
93         .build();
94 }
95
96 @GetMapping(value = @="/csrf-token") &Trandatdev89
97 public Map<String, String> getCsrfToken(CsrfToken csrfToken) {
98     Map<String, String> response = new HashMap<>();
99     response.put("csrfToken", csrfToken.getToken());
100     response.put("headerName", csrfToken.getHeaderName());
101     response.put("parameterName", csrfToken.getParameterName());
102     return
103     Map<String, String> response = new HashMap<String, String>()
104     skilline-server
105 }
106 }
```

b) Kiểm tra SameSite Cookie Attribute

Đặt cookie `SameSite=Strict` hoặc `SameSite=Lax`. Với `SameSite=Strict`, trình duyệt sẽ không gửi cookie khi request xuất phát từ domain khác, ngăn chặn hoàn toàn CSRF. Trong file `CookieUtil` (line 19,29)



```
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServletResponse;

public class CookieUtil {

    public static void setAccessTokenCookieHttpOnly(String accessToken, HttpServletResponse response) {
        Cookie accessTokenCookie = new Cookie(AppConstants.ACCESS_TOKEN, accessToken);
        accessTokenCookie.setHttpOnly(true);
        accessTokenCookie.setSecure(false);
        accessTokenCookie.setPath("/");
        accessTokenCookie.setMaxAge(30 * 24 * 60 * 60); //1 ngày
        accessTokenCookie.setAttribute(name: "SameSite", value: "Strict");
        response.addCookie(accessTokenCookie);
    }

    public static void setRefreshTokenCookieHttpOnly(String refreshTokenHttpOnly, HttpServletResponse response) {
        Cookie refreshTokenCookie = new Cookie(AppConstants.REFRESH_TOKEN, refreshTokenHttpOnly);
        refreshTokenCookie.setHttpOnly(true);
        refreshTokenCookie.setSecure(false);
        refreshTokenCookie.setPath("/");
        refreshTokenCookie.setMaxAge(30 * 24 * 60 * 60);
        refreshTokenCookie.setAttribute(name: "SameSite", value: "Strict");
        response.addCookie(refreshTokenCookie);
    }

    public static void setDeviceIdCookieHttpOnly(String currentDeviceId, HttpServletResponse response) {
        Cookie deviceIdCookie = new Cookie(name: "deviceId", currentDeviceId);
    }
}
```

c) Validate Origin / Referer Header

Backend kiểm tra `Origin` hoặc `Referer` header — chỉ chấp nhận request từ domain của ứng dụng. Đây là lớp bảo vệ bổ sung, không nên dùng đơn độc.

d) Yêu cầu xác nhận lại mật khẩu

Với các action nhạy cảm như đổi email, đổi mật khẩu — yêu cầu người dùng nhập lại mật khẩu hiện tại trước khi thực hiện. Đây là defense-in-depth hiệu quả ngay cả khi CSRF token bị bypass.

8. References

- OWASP CSRF Prevention Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- CWE-352: Cross-Site Request Forgery
- CVE-2023-34034 — Spring Security CSRF bypass (tham khảo)